

Synchronome Project — Raspberry Pi 3B+

Purpose

Build a synchronome that synchronizes with an external clock.

Hardware

Raspberry Pi 3B+, Logitech webcam (virtual clock via V4L2).

Constraint

Minimum of 2 services sharing one core.

Objectives

Minimum: Synchronize the synchronome over 3 minutes at 1 Hz. (clarify: image saving status for this configuration)

Maximum: Synchronize over 3 minutes at 10 Hz, with image saving.

Build Structure

- Preprocessor directives are used to build the minimum and maximum configurations.
- A frame-differencing function detects ticks: the consecutive-frame difference must fall between 20-40%. An upper bound is set to reject false positives and blurry frames.
- RM (Rate Monotonic) policy is used for priority assignment.

Preprocessor Directives

- **-min:** image saving, queue-only processing, 1 Hz
- **-target:** image saving, 10 Hz
- **-stretch:** no image saving, queue-only processing, 10 Hz, using a stopwatch/watch with a half-second hand

Core Allocation

- Core 1: Linux
- Core 2: T1 (Capture) + T3 (Diff)
- Core 3: T2 (Convert)
- Core 4: T4 (Write to SD card)

Shared Resources

- Frame queue between T1 and T2 — protected via the queue's own mechanism.
- Frame queue between T2 and T3 — protected via the queue's own mechanism.
- Frame queue between T3 and T4 — protected via the queue's own mechanism.

Feasibility

Five 1 Hz test runs were performed on a partial program, without FIFO scheduling or threading, to measure each service's execution time and feed the RMA calculation.

Cumulative results across 5 runs:

Service	Mean	Worst case (max)
CAPTURE	0.027 ms	0.130 ms
CONVERT	5.821 ms	11.966 ms
DIFF	0.948 ms	2.515 ms
WRITE	2.929 ms	60.342 ms

RMA Calculation

RMA is computed only for CAPTURE and DIFF, the only two services sharing a core.

- C_i = worst-case execution time
- T_i = period, derived from the webcam's maximum frame rate (≈ 29.8 Hz $\rightarrow \approx 33.5$ ms)
- n = number of services on the core

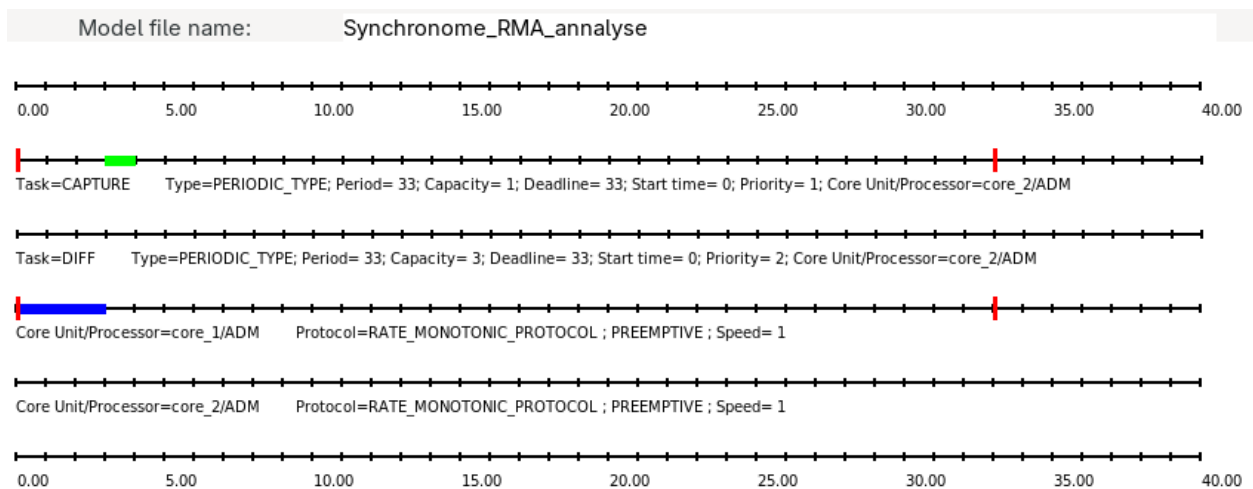
Formula: $U = \sum (C_i/T_i) \leq n \times (2^{(1/n)} - 1)$

$$U = (0.130/33.5) + (2.515/33.5) \approx 0.079$$

$$n \times (2^{(1/2)} - 1) = 2 \times (1.414 - 1) = 0.828$$

0.079 < 0.828 $\rightarrow$ Feasible

Scheduling diagram produced with Cheddar



RM Parameter Summary Table

Service	Ci (max)	Ti	Di	Core	Priority
T1 CAPTURE	0.130 ms	33.5 ms	33.5 ms	Core 2	Highest
T2 CONVERT	11.966 ms	33.5 ms	33.5 ms	Core 3	—
T3 DIFF	2.515 ms	33.5 ms	33.5 ms	Core 2	2nd (tie-break, see below)
T4 WRITE	60.342 ms	1000 ms	1000 ms	Core 4	—

Priority Tie-Break: T1 vs T3

T1 (CAPTURE) and T3 (DIFF) share Core 2 and have the same period ($T_i = 33.5$ ms), so pure Rate Monotonic assignment (priority by period) does not resolve which one should run at the higher priority. The tie is broken by worst-case execution time (C_i): the task with the shorter C_i is given the higher priority, since it releases the CPU faster and reduces overall system latency. T1 ($C_i \approx 0.130$ ms) is therefore assigned the higher priority over T3 ($C_i \approx 2.515$ ms).